

OOP Concepts Explained Simply

You said:

complete oops concept very easy word

ChatGPT said:

Complete OOP Concepts in Python — Very Easy Words

OOP means **Object-Oriented Programming**. It is a programming style in which we organize code using **classes and objects**.

Real-life example:

- Class = Car design/blueprint
- Object = Actual car created from that design
- Attributes = color, price, speed
- Methods = start(), stop(), accelerate()

1. Class

A **class** is a blueprint used to create objects.

```
class Student:  
    name = "Aamir"  
    course = "AI/ML"
```

Here, **Student** is a class.

2. Object

An **object** is an actual instance of a class.

```
class Student:  
    name = "Aamir"  
  
student1 = Student()  
  
print(student1.name)
```

Output:

```
Aamir
```

- **Student** = class
- **student1** = object
- **name** = attribute

3. Constructor: **__init__()**

A constructor initializes an object's data. It runs automatically when an object is created.

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

student1 = Student("Aamir", "AI/ML")
student2 = Student("Rahul", "Data Analytics")

print(student1.name)
print(student2.course)
```

Output:

```
Aamir
Data Analytics
```

4. What is **self**?

self represents the **current object**.

```
class Student:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print("My name is", self.name)

student1 = Student("Aamir")
student1.introduce()
```

Internally, Python treats this call:

```
student1.introduce()
```

like:

```
Student.introduce(student1)
```

self is not a keyword, but using the name **self** is the standard practice.

5. Attributes or Variables

Attributes store information about an object.

Instance variable

An instance variable belongs to a particular object.

```
class Student:
    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.name)
print(student2.name)
```

Each object has a different **name**.

Class variable

A class variable is shared by all objects.

```
class Student:
    college = "UEM Jaipur"

    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.college)
print(student2.college)
```

Output:

```
UEM Jaipur
UEM Jaipur
```

Class variable vs instance variable

Class variable	Instance variable
Belongs to class	Belongs to object
Shared by all objects	Separate for every object
Written outside methods	Usually written using <code>self</code>
Example: <code>college</code>	Example: <code>name</code>

6. Methods

A method is a function written inside a class.

Python has three important method types:

1. Instance method
2. Class method
3. Static method

Instance method

It works with object data and takes `self`.

```
class Student:
    def __init__(self, name):
        self.name = name

    def show_name(self):
        print(self.name)

student1 = Student("Aamir")
student1.show_name()
```

Class method

It works with class-level data and takes `cls`.


```
class Student:
    college = "UEM Jaipur"

    @classmethod
    def change_college(cls, new_college):
        cls.college = new_college

Student.change_college("IIT Delhi")

print(Student.college)
```

Output:

```
IIT Delhi
```

Static method

It does not need object data or class data. It works like a normal helper function placed inside a class.

```
class Calculator:
    @staticmethod
    def add(a, b):
        return a + b

print(Calculator.add(10, 20))
```

Output:

```
30
```

Method comparison

Method	First parameter	Works with
Instance method	<code>self</code>	Object data
Class method	<code>cls</code>	Class data

Method	First parameter	Works with
Static method	Nothing special	Utility logic

Four Pillars of OOP

The four main pillars are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

7. Encapsulation

Encapsulation means keeping data and the methods that work on that data inside one class.

It can also protect data from incorrect direct changes.

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount

    def show_balance(self):
        print("Balance:", self.balance)

account = BankAccount(1000)
account.deposit(500)
account.show_balance()
```

Output:

```
Balance: 1500
```

Public member

A public member can be accessed from anywhere.

```
class Student:
    def __init__(self, name):
        self.name = name

student = Student("Aamir")
print(student.name)
```

Protected member

A protected member uses one underscore `_`. It means: "Use this inside the class or child classes."

```
class Employee:
    def __init__(self):
        self._salary = 50000

employee = Employee()
print(employee._salary)
```

Python still allows access, but programmers should treat it as protected.

Private member

A private member uses two underscores `__`.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def show_balance(self):
        print(self.__balance)

account = BankAccount(1000)
account.show_balance()
```

This direct access will fail:

```
print(account.__balance)
```

Python changes its internal name using **name mangling**:

```
print(account._BankAccount__balance)
```

However, accessing private data this way is not recommended.

8. Getter and Setter

- Getter reads private data.
- Setter updates private data safely.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    def get_salary(self):
        return self.__salary

    def set_salary(self, salary):
        if salary > 0:
            self.__salary = salary
        else:
            print("Salary cannot be negative")

employee = Employee(50000)

print(employee.get_salary())
employee.set_salary(60000)
print(employee.get_salary())
```

9. Property Decorator

@property provides a cleaner way to use getters and setters.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, value):
        if value > 0:
            self.__salary = value
        else:
            print("Invalid salary")

employee = Employee(50000)

print(employee.salary)

employee.salary = 60000
print(employee.salary)
```

10. Inheritance

Inheritance means a child class receives properties and methods from a parent class.

```
class Person:
    def speak(self):
        print("Person can speak")

class Student(Person):
    def study(self):
        print("Student can study")

student = Student()

student.speak()
student.study()
```

Benefits:

- Code reuse
- Less duplicate code
- Easy maintenance

- Clear parent-child relationship

11. Types of Inheritance

Single inheritance

One child inherits from one parent.

```
class Parent:
    def show(self):
        print("Parent method")

class Child(Parent):
    pass

child = Child()
child.show()
```

Multilevel inheritance

A class inherits from a child class.

```
class Grandparent:
    def house(self):
        print("Grandparent's house")

class Parent(Grandparent):
    pass

class Child(Parent):
    pass

child = Child()
child.house()
```

Multiple inheritance

One child inherits from multiple parents.

```
class Father:
    def driving(self):
        print("Can drive")

class Mother:
    def cooking(self):
        print("Can cook")

class Child(Father, Mother):
    pass

child = Child()
child.driving()
child.cooking()
```

Hierarchical inheritance

Multiple child classes inherit from one parent.

```
class Animal:
    def eat(self):
        print("Animal is eating")

class Dog(Animal):
    pass

class Cat(Animal):
    pass

dog = Dog()
cat = Cat()

dog.eat()
cat.eat()
```

Hybrid inheritance

Hybrid inheritance is a combination of two or more inheritance types.

```
class A:  
    pass  
  
class B(A):  
    pass  
  
class C(A):  
    pass  
  
class D(B, C):  
    pass
```

12. **super()**

super() calls a method or constructor of the parent class.

```
class Person:  
    def __init__(self, name):  
        self.name = name  
  
class Student(Person):  
    def __init__(self, name, course):  
        super().__init__(name)  
        self.course = course  
  
    def show(self):  
        print(self.name, self.course)  
  
student = Student("Aamir", "AI/ML")  
student.show()
```

Output:

```
Aamir AI/ML
```

13. Polymorphism

Polymorphism means **one name, many forms**.

The same method name can behave differently for different objects.


```
class Dog:
    def sound(self):
        print("Dog barks")

class Cat:
    def sound(self):
        print("Cat meows")

dog = Dog()
cat = Cat()

dog.sound()
cat.sound()
```

14. Method Overriding

Method overriding happens when a child class provides its own version of a parent method.

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

dog = Dog()
dog.sound()
```

Output:

```
Dog barks
```

The child's method gets priority.

15. Method Overloading

Method overloading means having multiple methods with the same name but different parameters.

Python does not support traditional method overloading directly.

This will not work like Java:

```
class Calculator:  
    def add(self, a, b):  
        return a + b  
  
    def add(self, a, b, c):  
        return a + b + c
```

The second `add()` replaces the first one.

We can achieve similar behavior using default parameters:

```
class Calculator:  
    def add(self, a, b, c=0):  
        return a + b + c  
  
calculator = Calculator()  
  
print(calculator.add(10, 20))  
print(calculator.add(10, 20, 30))
```

Or using `*args`:

```
class Calculator:  
    def add(self, *numbers):  
        return sum(numbers)  
  
calculator = Calculator()  
  
print(calculator.add(10, 20))  
print(calculator.add(10, 20, 30, 40))
```

16. Duck Typing

Duck typing means Python focuses on what an object can do, not its exact class.

“If it behaves like a duck, treat it like a duck.”

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

def make_sound(animal):
    animal.sound()

make_sound(Dog())
make_sound(Cat())
```

The function works with any object that has a `sound()` method.

17. Operator Overloading

Operator overloading changes how operators work with custom objects.

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value

n1 = Number(10)
n2 = Number(20)

print(n1 + n2)
```

Output:

```
30
```

Here, `+` internally calls `__add__()`.

18. Abstraction

Abstraction means showing only the necessary information and hiding internal implementation.

Example: You use an ATM to withdraw money, but you do not need to know its internal code.

Python provides abstraction using the `abc` module.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):

    def start(self):
        print("Car starts with a key")

car = Car()
car.start()
```

We cannot create an object of an incomplete abstract class:

```
vehicle = Vehicle() # Error
```

19. Abstract Class and Abstract Method

Abstract class

A class created to provide a common structure to child classes.

Abstract method

A method declared in the parent class that child classes must implement.

```
from abc import ABC, abstractmethod

class Payment(ABC):

    @abstractmethod
    def pay(self, amount):
        pass

class UPI(Payment):

    def pay(self, amount):
        print(f"Paid ₹{amount} using UPI")

payment = UPI()
payment.pay(1000)
```

Object Relationships

20. Association

Association means two independent objects are connected.

Example: A teacher teaches a student, but both can exist separately.

```
class Teacher:
    def teach(self, student):
        print("Teaching", student.name)

class Student:
    def __init__(self, name):
        self.name = name

student = Student("Aamir")
teacher = Teacher()

teacher.teach(student)
```

21. Aggregation

Aggregation represents a weak “has-a” relationship.

The contained object can exist without the container.

Example: A department has teachers, but teachers can exist without the department.

```
class Teacher:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, teacher):
        self.teacher = teacher

teacher = Teacher("Mr. Sharma")
department = Department(teacher)

print(department.teacher.name)
```

The **Teacher** object was created outside the **Department**.

22. Composition

Composition represents a strong “has-a” relationship.

The contained object is created and controlled by the container.

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start_car(self):
        self.engine.start()
        print("Car started")

car = Car()
car.start_car()
```

The **Engine** is created inside the **Car**.

Aggregation vs composition

Aggregation	Composition
Weak relationship	Strong relationship
Object created outside	Object created inside
Child can exist independently	Child depends on container
Department has Teacher	Car has Engine

Advanced OOP Concepts

23. Method Resolution Order

MRO tells Python the order in which it searches classes for a method.

```
class A:
    def show(self):
        print("Class A")

class B(A):
    pass

class C(A):
    def show(self):
        print("Class C")

class D(B, C):
    pass

d = D()
d.show()

print(D.mro())
```

Python follows the order:

$D \rightarrow B \rightarrow C \rightarrow A \rightarrow \text{object}$

This helps Python solve confusion in multiple inheritance.

24. Magic or Dunder Methods

Magic methods begin and end with double underscores.

Examples:

- `__init__()` – constructor
- `__str__()` – readable object text
- `__len__()` – works with `len()`
- `__add__()` – works with `+`

- `__eq__()` – works with `==`

`__str__()`

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

    def __str__(self):
        return f"{self.name} is studying {self.course}"

student = Student("Aamir", "AI/ML")

print(student)
```

Output:

```
Aamir is studying AI/ML
```

`__len__()`

```
class Team:
    def __init__(self, members):
        self.members = members

    def __len__(self):
        return len(self.members)

team = Team(["Aamir", "Rahul", "Priya"])

print(len(team))
```

Output:

```
3
```


`__eq__()`

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    def __eq__(self, other):
        return self.marks == other.marks

s1 = Student(90)
s2 = Student(90)

print(s1 == s2)
```

Output:

```
True
```

25. Object Identity vs Equality

`is` checks whether two variables point to the same object.

`==` checks whether their values are equal.

```
list1 = [10, 20]
list2 = [10, 20]
list3 = list1

print(list1 == list2) # True
print(list1 is list2) # False
print(list1 is list3) # True
```

26. `isinstance()` and `issubclass()`

`isinstance()`

Checks whether an object belongs to a class.

```
class Student:
    pass

student = Student()

print(isinstance(student, Student))
```

Output:

```
True
```

issubclass()

Checks whether a class inherits from another class.

```
class Person:
    pass

class Student(Person):
    pass

print(issubclass(Student, Person))
```

Output:

```
True
```

27. Object Creation Process

When this line runs:

```
student = Student("Aamir")
```

Python performs these steps:

1. `__new__()` creates the object.
2. `__init__()` initializes its data.
3. The object reference is stored in `student`.

```
class Student:
    def __new__(cls, *args, **kwargs):
        print("Creating object")
        return super().__new__(cls)

    def __init__(self, name):
        print("Initializing object")
        self.name = name

student = Student("Aamir")
```

28. Destructor: `__del__()`

`__del__()` may run when an object is about to be destroyed.

```
class Student:
    def __init__(self, name):
        self.name = name
        print("Object created")

    def __del__(self):
        print("Object destroyed")

student = Student("Aamir")
```

Important: Do not depend on `__del__()` for important cleanup because its execution time is not always predictable. Context managers are better for files and resources.

29. Data Class

A data class reduces repeated constructor and display code.

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    course: str
    marks: int

student = Student("Aamir", "AI/ML", 90)

print(student)
```

Output:

```
Student(name='Aamir', course='AI/ML', marks=90)
```

30. Complete Practical Example

```
from abc import ABC, abstractmethod

class Employee(ABC):
    company = "Tech Solutions"

    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary")

    @abstractmethod
    def calculate_bonus(self):
        pass

    def show_details(self):
        print("Name:", self.name)
        print("Company:", self.company)
        print("Salary:", self.salary)

class Developer(Employee):

    def __init__(self, name, salary, language):
        super().__init__(name, salary)
        self.language = language

    def calculate_bonus(self):
        return self.salary * 0.10

    def show_details(self):
        super().show_details()
        print("Language:", self.language)
        print("Bonus:", self.calculate_bonus())

developer = Developer("Aamir", 50000, "Python")
developer.show_details()
```

This example contains:

- Class and object
- Constructor
- Instance and class variables
- Encapsulation
- Property
- Abstraction
- Inheritance
- Method overriding
- Polymorphism
- `super()`

Most Important Interview Questions

1. What is OOP?

OOP is a programming style that organizes code using classes and objects.

2. What is a class?

A class is a blueprint used to create objects.

3. What is an object?

An object is an instance of a class containing data and behavior.

4. What are the four pillars of OOP?

Encapsulation, inheritance, polymorphism and abstraction.

5. What is encapsulation?

Encapsulation keeps data and related methods together and controls how data is accessed.

6. What is inheritance?

Inheritance allows a child class to reuse properties and methods from a parent class.

7. What is polymorphism?

Polymorphism allows the same method or operation to behave differently for different objects.

8. What is abstraction?

Abstraction hides internal implementation and shows only necessary functionality.

9. What is **self**?

self represents the current object of a class.

10. What is **super()**?

super() is used to access methods or constructors of the parent class.

11. Overloading vs overriding?

Overloading	Overriding
Same method, different parameters	Child changes parent method
Happens in the same class	Happens between parent and child
Not directly supported in Python	Supported in Python

12. Class vs object?

Class	Object
Blueprint	Actual instance
Logical structure	Real entity in memory
<code>class Student:</code>	<code>s1 = Student()</code>

13. Encapsulation vs abstraction?

Encapsulation	Abstraction
Protects data	Hides implementation
Controls data access	Shows essential features
Uses private members/property	Uses abstract classes/methods

14. Inheritance vs composition?

- Inheritance: **is-a** relationship — Developer is an Employee.
- Composition: **has-a** relationship — Car has an Engine.

15. Why is OOP useful?

- Reuses code

- Reduces duplication
- Protects data
- Makes code easier to maintain
- Represents real-world entities
- Helps build large applications

Quick Revision

Class → Blueprint
Object → Instance of a class
Attribute → Data of an object
Method → Function inside a class
Constructor → Initializes an object
self → Current object
Encapsulation → Protect data
Inheritance → Reuse parent code
Polymorphism → One interface, many behaviors
Abstraction → Hide implementation
super() → Access parent functionality
MRO → Method-search order
Composition → Strong has-a relationship
Aggregation → Weak has-a relationship

You said:

Class → Blueprint Object → Instance of a class Attribute → Data of an object Method → Function inside a class Constructor → Initializes an object self → Current object Encapsulation → Protect data Inheritance → Reuse parent code Polymorphism → One interface, many behaviors Abstraction → Hide implementation super() → Access parent functionality MRO → Method-search order Composition → Strong has-a relationship Aggregation → Weak has-a relationship explain for interview

ChatGPT said:

OOP Concepts for Interview — Easy Answers

1. Class — Blueprint

A **class** is a blueprint or template used to create objects. It defines what data and functions its objects will have.

Real-life example: A house map is a blueprint, while actual houses are objects.

```
class Student:
    def study(self):
        print("Student is studying")
```

Interview answer:

“A class is a user-defined blueprint that groups related data and methods in one place.”

2. Object — Instance of a Class

An **object** is a real instance created from a class. Every object can have its own data.

```
class Student:
    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.name)
print(student2.name)
```

Here:

- **Student** is the class.
- **student1** and **student2** are objects.

Interview answer:

“An object is an instance of a class that contains actual data and can use the class methods.”

3. Attribute — Data of an Object

An **attribute** is a variable that stores information about an object.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

student = Student("Aamir", 90)

print(student.name)
print(student.marks)
```


name and **marks** are attributes.

There are two main types:

- Instance attribute: Different for every object
- Class attribute: Shared by all objects

```
class Student:
    college = "UEM Jaipur"    # Class attribute

    def __init__(self, name):
        self.name = name      # Instance attribute
```

Interview answer:

“An attribute is data associated with a class or object, such as a student's name or marks.”

4. Method — Function Inside a Class

A **method** is a function defined inside a class. It represents the behavior of an object.

```
class Student:
    def study(self):
        print("Student is studying")

student = Student()
student.study()
```

Here, **study()** is a method.

Main method types:

- Instance method
- Class method
- Static method

Interview answer:

“A method is a function inside a class that defines what an object can do.”

5. Constructor — Initializes an Object

A constructor runs automatically when an object is created. In Python, the constructor is usually **__init__()**.

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

student = Student("Aamir", "AI/ML")

print(student.name)
```

Output:

```
Aamir
```

Interview answer:

“A constructor is a special method used to initialize the attributes of a newly created object. In Python, we use `__init__()`.”

Important point: Technically, `__new__()` creates the object and `__init__()` initializes it.

6. `self` — Current Object

`self` represents the current object that is calling the method.

```
class Student:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print("My name is", self.name)

student = Student("Aamir")
student.introduce()
```

Python internally treats:

```
student.introduce()
```

like:

```
Student.introduce(student)
```

Interview answer:

“**self** refers to the current object. It is used to access that object's attributes and methods.”

Important point: **self** is not a Python keyword, but it is the standard name.

Four Pillars of OOP

7. Encapsulation — Protect Data

Encapsulation means keeping data and related methods inside one class. It also controls how data is accessed or modified.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)

print(account.get_balance())
```

Output:

```
1500
```

__balance is treated as a private attribute.

Benefits:

- Protects data
- Prevents invalid updates
- Makes code safer
- Controls access through methods

Interview answer:

“Encapsulation means combining data and methods inside a class and restricting direct access to sensitive data.”

Important Python point: Python uses naming conventions and name mangling rather than strict private access like Java.

8. Inheritance — Reuse Parent Code

Inheritance allows a child class to use the attributes and methods of a parent class.

```
class Person:
    def speak(self):
        print("Person can speak")

class Student(Person):
    def study(self):
        print("Student can study")

student = Student()

student.speak()
student.study()
```

Here:

- **Person** is the parent class.
- **Student** is the child class.
- **Student** inherits **speak()** from **Person**.

Benefits:

- Code reuse
- Less duplicate code
- Easy maintenance
- Supports method overriding

Interview answer:

“Inheritance allows a child class to reuse and extend the features of a parent class. It represents an ‘is-a’ relationship.”

Example: A Student **is a** Person.

Types of inheritance:

- Single
- Multiple
- Multilevel
- Hierarchical

- Hybrid

9. Polymorphism — One Interface, Many Behaviors

Polymorphism means the same method name can behave differently for different objects.

```
class Dog:
    def sound(self):
        print("Dog barks")

class Cat:
    def sound(self):
        print("Cat meows")

def make_sound(animal):
    animal.sound()

make_sound(Dog())
make_sound(Cat())
```

Output:

```
Dog barks
Cat meows
```

The same `sound()` call produces different results.

Common forms in Python:

- Method overriding
- Duck typing
- Operator overloading
- Default arguments or `*args`

Interview answer:

“Polymorphism means one interface can have multiple implementations. The same method call can behave differently depending on the object.”

Method overriding example

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")
```

The child class replaces the parent's method behavior.

10. Abstraction — Hide Implementation

Abstraction means hiding unnecessary internal details and showing only important functionality.

Real-life example: We drive a car using the steering wheel and pedals without knowing the engine's complete internal working.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):

    def start(self):
        print("Car starts with a key")

car = Car()
car.start()
```

The **Vehicle** class defines what a vehicle must do. The **Car** class decides how it will do it.

Interview answer:

“Abstraction hides internal implementation and exposes only essential functionality. In Python, it can be implemented using abstract classes and abstract methods.”

Important point: We cannot create an object of an incomplete abstract class.

11. `super()` — Access Parent Functionality

`super()` is used inside a child class to call the parent class's constructor or methods.

```
class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)

class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

    def show(self):
        super().show()
        print("Course:", self.course)

student = Student("Aamir", "AI/ML")
student.show()
```

Output:

```
Name: Aamir
Course: AI/ML
```

Benefits:

- Reuses parent logic
- Avoids duplicate code
- Useful in multiple inheritance
- Follows Python's MRO

Interview answer:

“`super()` provides access to the next parent class according to the method resolution order. It is commonly used to call the parent constructor or method.”

12. MRO — Method-Search Order

MRO means **Method Resolution Order**. It tells Python the order in which classes should be searched for a method.

```
class A:
    def show(self):
        print("Class A")

class B(A):
    pass

class C(A):
    def show(self):
        print("Class C")

class D(B, C):
    pass

obj = D()
obj.show()

print(D.mro())
```

Output:

```
Class C
[D, B, C, A, object]
```

Python searches in this order:

```
D → B → C → A → object
```

MRO is especially important in multiple inheritance and solves the diamond problem.

Interview answer:

“MRO defines the order in which Python searches parent classes for a method. Python uses C3 linearization to calculate this order.”

Check MRO using:

```
ClassName.mro()
```

or:

```
ClassName.__mro__
```


13. Composition — Strong “Has-a” Relationship

Composition means one object strongly owns another object. The contained object is usually created inside the owner class.

Example: A car has an engine.

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
        print("Car started")

car = Car()
car.start()
```

Here, the **Engine** is created and controlled by the **Car**.

Interview answer:

“Composition is a strong ‘has-a’ relationship where one object owns another object, and the contained object usually depends on the owner.”

Examples:

- Car has an Engine
- House has Rooms
- Computer has a Processor

14. Aggregation — Weak “Has-a” Relationship

Aggregation is also a “has-a” relationship, but both objects can exist independently.

Example: A department has teachers.

```

class Teacher:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, teacher):
        self.teacher = teacher

teacher = Teacher("Mr. Sharma")
department = Department(teacher)

print(department.teacher.name)

```

The **Teacher** object is created outside the **Department**. Therefore, the teacher can exist even without that department.

Interview answer:

“Aggregation is a weak ‘has-a’ relationship where the contained object can exist independently of the container object.”

Examples:

- Department has Teachers
- Team has Players
- Library has Books

Composition vs Aggregation

Composition	Aggregation
Strong relationship	Weak relationship
Object created inside owner	Object usually passed from outside
Contained object depends on owner	Both objects can exist independently
Car has Engine	Department has Teacher
Strong ownership	Weak ownership

Final Interview Revision

Concept	Easy interview meaning
Class	Blueprint used to create objects

Concept	Easy interview meaning
Object	Actual instance of a class
Attribute	Data belonging to a class or object
Method	Function defined inside a class
Constructor	Initializes a new object
<code>self</code>	Refers to the current object
Encapsulation	Protects and controls data access
Inheritance	Reuses and extends parent code
Polymorphism	Same interface, different behaviors
Abstraction	Hides implementation details
<code>super()</code>	Accesses parent functionality through MRO
MRO	Order for searching inherited methods
Composition	Strong “has-a” relationship
Aggregation	Weak “has-a” relationship

You said:

Class → Blueprint Object → Instance of a class Attribute → Data of an object Method → Function inside a class Constructor → Initializes an object self → Current object Encapsulation → Protect data Inheritance → Reuse parent code Polymorphism → One interface, many behaviors Abstraction → Hide implementation super() → Access parent functionality MRO → Method-search order Composition → Strong has-a relationship Aggregation → Weak has-a relationship with real life example all types and concept explain and every word to give easy mean

ChatGPT said:

Complete OOP in Python — Easy Meaning, Types and Real-Life Examples

First understand these common words

Word	Easy meaning	Example
Blueprint	A plan used to create something	House map
Instance	An actual item created from a blueprint	Actual house
Data	Stored information	Name, age, salary
Behaviour	An action something can perform	Start, stop, study
Initialize	Give starting values	Set initial balance to ₹1,000
Interface	The visible way to use something	ATM buttons
Implementation	Internal logic that performs the work	ATM's internal software
Parent class	Class whose features are inherited	Person
Child class	Class that inherits features	Student
Reuse	Use existing code again	Child uses parent method
Access	Read or use something	Reading <code>student.name</code>
Override	Replace inherited behaviour	Dog changes <code>sound()</code>
Has-a	One object contains another	Car has an engine
Is-a	One object is a type of another	Student is a person

1. Class — Blueprint

Easy meaning

A **class** is a plan or blueprint used to create objects.

A class tells Python:

- What information an object will store
- What actions an object can perform

Real-life example

A house map describes:

- Number of rooms
- Number of doors
- House colour
- House size

The map is not an actual house. It is only a plan.

Similarly:

- **Car** class = car design
- Actual BMW car = object
- **Student** class = student structure
- Aamir = object

Python example

```
class Student:  
    def study(self):  
        print("Student is studying")
```

Here:

- **class** is a Python keyword used to create a class.
- **Student** is the class name.
- **study()** is a method.

Creating objects from a class

```
student1 = Student()  
student2 = Student()  
  
student1.study()  
student2.study()
```

student1 and **student2** are two different objects of the same class.

Common class types

Concrete class

A normal class from which objects can be created.

```
class Student:  
    pass  
  
student = Student()
```

Parent class

A class whose features are inherited by another class.

```
class Person:  
    pass
```

Child class

A class that inherits from another class.

```
class Student(Person):  
    pass
```

Abstract class

A class that provides a common structure but may not allow direct object creation.

```
from abc import ABC  
  
class Vehicle(ABC):  
    pass
```

Interview answer

“A class is a user-defined blueprint that combines related attributes and methods. Objects are created from the class.”

2. Object — Instance of a Class

Easy meaning

An **object** is an actual item created from a class.

An object is also called an **instance** of a class.

Real-life example

Suppose **Car** is a class.

Actual cars can be:

- A red BMW
- A white Tata Nexon
- A black Mahindra Thar

All are objects of the **Car** class, but each car can have different data.

Python example

```
class Car:
    def __init__(self, brand, colour):
        self.brand = brand
        self.colour = colour

car1 = Car("BMW", "Black")
car2 = Car("Tata", "White")

print(car1.brand, car1.colour)
print(car2.brand, car2.colour)
```

Output:

```
BMW Black
Tata White
```

Every object has three important things

Identity

Identity tells us which exact object it is.

```
print(id(car1))  
print(id(car2))
```

Both objects normally have different identities.

State

State means the current data of an object.

For `car1`, the state is:

```
brand = BMW  
colour = Black
```

Behaviour

Behaviour means the actions an object can perform.

```
class Car:  
    def start(self):  
        print("Car started")
```

Here, `start()` represents behaviour.

Check an object's class

```
print(isinstance(car1, Car))
```

Output:

```
True
```

Interview answer

“An object is an instance of a class. It has identity, state and behaviour. For example, a specific BMW is an object of the Car class.”

3. Attribute — Data of an Object

Easy meaning

An **attribute** is a variable that stores information about a class or object.

Real-life example

A student has:

- Name
- Roll number
- Course
- Marks

These are the student's attributes.

Python example

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

student = Student("Aamir", 90)

print(student.name)
print(student.marks)
```

Here, **name** and **marks** are attributes.

Types of attributes

1. Instance attribute

An instance attribute belongs to a particular object. Different objects can have different values.

```
class Student:
    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.name)
print(student2.name)
```

Output:

```
Aamir
Rahul
```

name is an instance attribute because every student can have a different name.

2. Class attribute

A class attribute belongs to the class and is shared by all objects.

```
class Student:
    college = "UEM Jaipur"

    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.college)
print(student2.college)
```

Output:

```
UEM Jaipur
UEM Jaipur
```

Class vs instance attribute

Class attribute

Instance attribute

Belongs to the class

Belongs to an object

Class attribute	Instance attribute
Shared by all objects	Separate for each object
Defined directly in class	Usually defined using <code>self</code>
Example: college name	Example: student name

3. Dynamic attribute

Python allows us to add an attribute after object creation.

```
student.city = "Jaipur"

print(student.city)
```

This is possible, but important attributes should normally be defined inside `__init__()`.

Interview answer

“An attribute is data associated with a class or object. Instance attributes are separate for every object, while class attributes are shared.”

4. Method — Function Inside a Class

Easy meaning

A **method** is a function written inside a class. It defines what an object or class can do.

Real-life example

A mobile phone has data:

- Brand
- Model
- Battery

It can perform actions:

- Call
- Charge
- Switch off

These actions are methods.

Python example

```
class Mobile:
    def call(self):
        print("Calling someone")

mobile = Mobile()
mobile.call()
```

Types of methods

1. Instance method

An instance method works with object data and takes **self** as its first parameter.

```
class Student:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print("My name is", self.name)

student = Student("Aamir")
student.introduce()
```

2. Class method

A class method works with class-level data. It takes **cls** as its first parameter.

```
class Student:
    college = "UEM Jaipur"

    @classmethod
    def change_college(cls, new_college):
        cls.college = new_college

Student.change_college("Delhi University")

print(Student.college)
```

@classmethod is called a **decorator**. It tells Python that the following function is a class method.

3. Static method

A static method is a utility function related to the class. It does not need **self** or **cls**.

```
class Calculator:  
    @staticmethod  
    def add(a, b):  
        return a + b  
  
print(Calculator.add(10, 20))
```

4. Magic method

Magic methods begin and end with double underscores.

Examples:

```
__init__()  
__str__()  
__len__()  
__add__()
```

```
class Student:  
    def __init__(self, name):  
        self.name = name  
  
    def __str__(self):  
        return f"Student name: {self.name}"  
  
student = Student("Aamir")  
print(student)
```

5. Abstract method

An abstract method defines what a child class must implement.

```
from abc import ABC, abstractmethod  
  
class Vehicle(ABC):  
  
    @abstractmethod  
    def start(self):  
        pass
```

Method comparison

Method	First parameter	Main purpose
Instance method	<code>self</code>	Work with object data
Class method	<code>cls</code>	Work with class data
Static method	Nothing special	Perform utility work
Magic method	Depends on method	Support built-in operations
Abstract method	Usually <code>self</code>	Force child implementation

Interview answer

“A method is a function defined inside a class. Python supports instance methods, class methods, static methods, magic methods and abstract methods.”

5. Constructor — Initializes an Object

Easy meaning

A constructor gives starting values to a new object.

Initialize means setting the object's initial or beginning data.

Real-life example

When a new bank account is opened, the bank sets:

- Customer name
- Account number
- Opening balance

This is similar to object initialization.

Python example

```
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

account = BankAccount("Aamir", 1000)

print(account.name)
print(account.balance)
```

`__init__()` runs automatically when the object is created.

Types of constructors commonly discussed

1. Non-parameterized constructor

It does not take custom object data.

```
class Student:
    def __init__(self):
        self.name = "Unknown"

student = Student()
print(student.name)
```

2. Parameterized constructor

It receives values when the object is created.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

student = Student("Aamir", 90)
```

3. Constructor with default values

```
class Student:
    def __init__(self, name="Unknown", marks=0):
        self.name = name
        self.marks = marks

student1 = Student()
student2 = Student("Aamir", 90)
```

4. Alternative constructor using a class method

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    @classmethod
    def from_string(cls, information):
        name, marks = information.split("-")
        return cls(name, int(marks))

student = Student.from_string("Aamir-90")

print(student.name)
print(student.marks)
```

Important technical point

In common interview language, `__init__()` is called the constructor. Technically:

- `__new__()` creates the object.
- `__init__()` initializes the object.

```
class Student:
    def __new__(cls, *args, **kwargs):
        print("Object is being created")
        return super().__new__(cls)

    def __init__(self, name):
        print("Object is being initialized")
        self.name = name

student = Student("Aamir")
```


Interview answer

“A constructor is a special method that initializes a newly created object. In Python, `__init__()` is commonly called the constructor.”

6. `self` — Current Object

Easy meaning

`self` refers to the current object using a method.

Real-life example

Suppose there are two students:

- Aamir
- Rahul

When Aamir introduces himself, “my name” means Aamir’s name. When Rahul introduces himself, “my name” means Rahul’s name.

`self` works like the word “my” for the current object.

Python example

```
class Student:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print("My name is", self.name)

student1 = Student("Aamir")
student2 = Student("Rahul")

student1.introduce()
student2.introduce()
```

Output:

```
My name is Aamir
My name is Rahul
```

For `student1.introduce()`, `self` refers to `student1`.

For `student2.introduce()`, `self` refers to `student2`.

Internal working

```
student1.introduce()
```

Python internally treats it approximately as:

```
Student.introduce(student1)
```

Important points

- `self` is the first parameter of an instance method.
- Python passes the current object automatically.
- `self` is not a reserved Python keyword.
- We could use another name, but `self` is the standard convention.

Interview answer

“`self` represents the current object. It is used to access that object’s attributes and methods inside the class.”

7. Encapsulation — Protect and Control Data

Easy meaning

Encapsulation means:

1. Keeping data and related methods together inside a class
2. Controlling how the data is accessed and changed

Real-life example

A bank account balance should not be changed directly by anyone.

The balance should be updated through controlled operations such as:

- Deposit
- Withdraw
- Transfer

The bank checks all rules before changing the balance.

Python example

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
        else:
            print("Invalid deposit amount")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Invalid withdrawal")

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
account.withdraw(200)

print(account.get_balance())
```

Output:

```
1300
```

Access levels in Python

1. Public member

A public member can be accessed from anywhere.

```
class Student:
    def __init__(self, name):
        self.name = name

student = Student("Aamir")
print(student.name)
```

2. Protected member

A protected member starts with one underscore `_`.

```
class Employee:
    def __init__(self, salary):
        self._salary = salary
```

It means: "This value is intended for internal use or child classes."

Python still allows this:

```
employee = Employee(50000)
print(employee._salary)
```

The underscore is mainly a programmer convention.

3. Private member

A private member starts with two underscores `__`.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance
```

Direct access normally fails:

```
account = BankAccount(1000)
print(account.__balance) # AttributeError
```

Python uses **name mangling**, changing the internal name approximately to:

```
__BankAccount__balance
```

It improves protection but does not provide absolute security.

Getter and setter

- Getter = reads a value

- Setter = changes a value after validation

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    def get_salary(self):
        return self.__salary

    def set_salary(self, salary):
        if salary > 0:
            self.__salary = salary
        else:
            print("Salary must be positive")
```

Property decorator

@property provides a cleaner getter and setter style.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, value):
        if value > 0:
            self.__salary = value
        else:
            print("Invalid salary")

employee = Employee(50000)

print(employee.salary)

employee.salary = 60000
print(employee.salary)
```

Benefits

- Protects important data
- Validates new values
- Prevents incorrect updates
- Keeps code organized

- Makes internal changes easier

Interview answer

“Encapsulation means combining data and methods inside a class and controlling direct access to the data. In Python, we use public, protected and private naming conventions, along with properties.”

8. Inheritance — Reuse Parent Code

Easy meaning

Inheritance allows a new class to reuse and extend the features of an existing class.

- Existing class = parent, base or superclass
- New class = child, derived or subclass

Real-life example

A student is a person.

Every person may have:

- Name
- Age
- Address

A student additionally has:

- Roll number
- Course
- Marks

The **Student** class can reuse common features from **Person**.

```

class Person:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(self.name, "can speak")

class Student(Person):
    def study(self):
        print(self.name, "is studying")

student = Student("Aamir")

student.speak()
student.study()

```

Types of inheritance

1. Single inheritance

One child inherits from one parent.

flowchart TD
A["Person"] --> B["Student"]

```

class Person:
    def speak(self):
        print("Speaking")

class Student(Person):
    def study(self):
        print("Studying")

```

2. Multilevel inheritance

Inheritance happens in multiple levels.

flowchart TD
A["Person"] --> B["Employee"]
B --> C["Manager"]

```
class Person:
    def speak(self):
        print("Person speaks")

class Employee(Person):
    def work(self):
        print("Employee works")

class Manager(Employee):
    def manage(self):
        print("Manager manages a team")

manager = Manager()

manager.speak()
manager.work()
manager.manage()
```

3. Multiple inheritance

One child inherits from two or more parents.

```
flowchart TD
    A["Father"] --> C["Child"]
    B["Mother"] --> C
```

```
class Father:
    def drive(self):
        print("Can drive")

class Mother:
    def cook(self):
        print("Can cook")

class Child(Father, Mother):
    pass

child = Child()
child.drive()
child.cook()
```

4. Hierarchical inheritance

Multiple children inherit from one parent.

flowchart TD

A["Employee"] --> B["Developer"]

A --> C["Tester"]

A --> D["Manager"]

```
class Employee:
    def work(self):
        print("Employee works")

class Developer(Employee):
    pass

class Tester(Employee):
    pass

developer = Developer()
tester = Tester()

developer.work()
tester.work()
```

5. Hybrid inheritance

Hybrid inheritance is a combination of multiple inheritance types.

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```

This combines hierarchical and multiple inheritance.

Method overriding

A child class can replace an inherited method.

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

dog = Dog()
dog.sound()
```

Output:

Dog barks

Benefits

- Reuses code
- Reduces duplication
- Supports extension
- Makes related classes organized
- Supports polymorphism

Interview answer

“Inheritance allows a child class to reuse and extend the attributes and methods of a parent class. It represents an ‘is-a’ relationship.”

9. Polymorphism — One Interface, Many Behaviours

Easy meaning

- **Poly** means many.
- **Morphism** means forms.
- Polymorphism means many forms.

The same method name or operation can behave differently for different objects.

Real-life example

The action `pay()` can work differently:

- UPI payment uses a PIN.
- Card payment uses card details.

- Cash payment uses physical money.

The operation is the same—payment—but the behaviour is different.

Python example

```
class UPI:
    def pay(self, amount):
        print(f"Paid ₹{amount} using UPI")

class Card:
    def pay(self, amount):
        print(f"Paid ₹{amount} using Card")

def complete_payment(payment_method, amount):
    payment_method.pay(amount)

complete_payment(UPI(), 1000)
complete_payment(Card(), 1000)
```

Types of polymorphism in Python

1. Method overriding

The child provides a different implementation of a parent method.

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Bark")
```

2. Duck typing

Python checks whether an object supports the required operation rather than checking its exact class.

“If an object behaves like a duck, we can treat it like a duck.”

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

def make_sound(animal):
    animal.sound()

make_sound(Dog())
make_sound(Cat())
```

The function accepts any object that provides `sound()`.

3. Operator overloading

We can define how an operator works with custom objects.

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __str__(self):
        return f"₹{self.amount}"

money1 = Money(500)
money2 = Money(300)

print(money1 + money2)
```

Output:

```
₹800
```

+ calls the `__add__()` method.

4. Method overloading-like behaviour

Traditional method overloading means using the same method name with different parameter lists.

Python does not support traditional method overloading directly. A later method replaces an earlier method with the same name.

We use default parameters:

```
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c

calculator = Calculator()

print(calculator.add(10, 20))
print(calculator.add(10, 20, 30))
```

Or ***args**:

```
class Calculator:
    def add(self, *numbers):
        return sum(numbers)

calculator = Calculator()

print(calculator.add(10, 20))
print(calculator.add(10, 20, 30, 40))
```

5. Built-in polymorphism

The same built-in function works with different data types.

```
print(len("Python"))
print(len([10, 20, 30]))
print(len({"name": "Aamir", "course": "AI/ML"}))
```

len() works differently with strings, lists and dictionaries.

Interview answer

“Polymorphism means one interface can have multiple implementations. The same method call can produce different behaviour depending on the object.”

10. Abstraction — Hide Internal Implementation

Easy meaning

Abstraction means:

- Show what an object can do.
- Hide how it performs that work internally.

Real-life example

When using an ATM, you only see:

- Insert card
- Enter PIN
- Select amount
- Withdraw cash

You do not see:

- Database queries
- Account verification code
- Banking server communication
- Cash-counting logic

The necessary interface is visible, while internal implementation is hidden.

Abstract class

An abstract class provides a common structure for child classes.

Abstract method

An abstract method declares a required operation but leaves its implementation to child classes.

```
from abc import ABC, abstractmethod

class Payment(ABC):

    @abstractmethod
    def pay(self, amount):
        pass
```

A child must implement `pay()`:

```
class UPI(Payment):
    def pay(self, amount):
        print(f"Paid ₹{amount} through UPI")

class Card(Payment):
    def pay(self, amount):
        print(f"Paid ₹{amount} through Card")

upi = UPI()
upi.pay(1000)
```

Concrete class

A concrete class implements all required abstract methods and allows object creation.

UPI is a concrete class.

Can we create an abstract class object?

No, not if it contains an unimplemented abstract method.

```
payment = Payment() # TypeError
```

Interface concept in Python

Python does not have a separate **interface** keyword like Java. An abstract class containing only abstract methods can be used like an interface.

```
from abc import ABC, abstractmethod

class Printable(ABC):

    @abstractmethod
    def print_data(self):
        pass
```

Encapsulation vs abstraction

Encapsulation	Abstraction
Protects and controls data	Hides internal implementation
Focuses on data access	Focuses on essential functionality

Encapsulation	Abstraction
Uses private data and properties	Uses abstract classes and methods
Example: Protect bank balance	Example: Hide ATM internal logic

Interview answer

“Abstraction hides implementation details and exposes only essential functionality. In Python, it is commonly implemented using the `ABC` class and `@abstractmethod` decorator.”

11. `super()` — Access Parent Functionality

Easy meaning

`super()` gives a child class access to the next parent class according to Python's MRO.

It is commonly used to call:

- Parent constructor
- Parent method

Real-life example

A student is also a person. Instead of rewriting the logic for setting a person's name, the student can call the parent's constructor.

```
class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)

class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

    def show(self):
        super().show()
        print("Course:", self.course)

student = Student("Aamir", "AI/ML")
student.show()
```


Output:

```
Name: Aamir  
Course: AI/ML
```

Without `super()`

We can technically call the parent directly:

```
Person.__init__(self, name)
```

However, `super()` is generally better because it works according to MRO and supports cooperative multiple inheritance.

Important point

`super()` does not always mean “direct parent.” It means “go to the next class in the MRO.”

Interview answer

“`super()` is used to access the next parent implementation according to MRO. It helps reuse parent constructors and methods without directly naming the parent class.”

12. MRO — Method Resolution Order

Easy meaning

MRO means **Method Resolution Order**.

It tells Python the order in which classes will be searched for an attribute or method.

Why is it needed?

Suppose one class inherits from multiple classes, and several parent classes contain a method with the same name.

Python needs a fixed rule to decide which method should run.

Simple example

```
class A:
    def show(self):
        print("Class A")

class B(A):
    def show(self):
        print("Class B")

class C(A):
    def show(self):
        print("Class C")

class D(B, C):
    pass

obj = D()
obj.show()
```

Output:

```
Class B
```

Python searches in this order:

```
D → B → C → A → object
```

Check the MRO:

```
print(D.mro())
```

or:

```
print(D.__mro__)
```

Diamond problem

The diamond problem appears when two parent classes inherit from the same grandparent.

flowchart TD

```
A["A: Grandparent"] --> B["B: Parent 1"]
A --> C["C: Parent 2"]
B --> D["D: Child"]
C --> D
```

Python uses MRO to ensure a predictable order and avoid repeatedly searching the same parent.

C3 linearization

Python calculates MRO using an algorithm called **C3 linearization**.

Easy meaning: It creates a consistent order that:

- Checks the child before parents
- Preserves the written parent order
- Avoids checking the same class repeatedly
- Keeps inheritance order consistent

MRO and **super()**

super() follows MRO.

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")
        super().show()

class C(A):
    def show(self):
        print("C")
        super().show()

class D(B, C):
    def show(self):
        print("D")
        super().show()

D().show()
```

Output:

D
B
C
A

Interview answer

“MRO is the order in which Python searches classes for a method or attribute. It is especially important in multiple inheritance, and Python calculates it using C3 linearization.”

13. Composition — Strong “Has-a” Relationship

Easy meaning

Composition means one object strongly owns or contains another object.

The contained object is usually created inside the owner.

Real-life example

A car has an engine.

A car needs its engine as an internal part. The **Car** class creates and controls the **Engine**.

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
        print("Car started")

car = Car()
car.start()
```

Why is it called strong?

The owner controls the contained object.

In the example:

- Car creates the Engine.
- Car stores the Engine.
- Engine is treated as a part of Car.

More real-life examples

- House has rooms
- Human has a heart
- Computer has a processor
- Order has order items

Composition vs inheritance

- Inheritance represents **is-a**.
- Composition represents **has-a**.

Examples:

- Developer is an Employee → inheritance
- Car has an Engine → composition

Interview answer

“Composition is a strong has-a relationship where one object owns and controls another object. The contained object is usually created inside the container.”

14. Aggregation — Weak “Has-a” Relationship

Easy meaning

Aggregation means one object uses or contains another object, but both objects can exist independently.

Real-life example

A department has teachers.

A teacher can exist without a particular department. If the department closes, the teacher may join another department.

```
class Teacher:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, name, teacher):
        self.name = name
        self.teacher = teacher

teacher = Teacher("Mr. Sharma")

department = Department("Computer Science", teacher)

print(department.name)
print(department.teacher.name)
```

Here:

- The **Teacher** object is created outside.
- The **Teacher** object is passed to **Department**.
- Both objects can exist independently.

More real-life examples

- Team has players
- Library has books
- School has teachers
- Company has employees

Interview answer

“Aggregation is a weak has-a relationship where one object uses another, but both can exist independently.”

Composition vs Aggregation

Point	Composition	Aggregation
Relationship	Strong has-a	Weak has-a
Ownership	Strong ownership	Weak ownership
Object creation	Usually created inside	Usually created outside
Independence	Contained object depends on owner	Both objects can exist separately

Point	Composition	Aggregation
Example	Car has an engine	Department has teachers

Code difference

Composition: object created inside

```
class Car:  
    def __init__(self):  
        self.engine = Engine()
```

Aggregation: object passed from outside

```
teacher = Teacher("Mr. Sharma")  
department = Department(teacher)
```

Complete Real-Life Example

This example combines most OOP concepts:

```
from abc import ABC, abstractmethod

class Person:
    organization = "Tech Solutions"

    def __init__(self, name):
        self.name = name

    def show_details(self):
        print("Name:", self.name)

class Employee(Person, ABC):
    def __init__(self, name, salary):
        super().__init__(name)
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary")

    @abstractmethod
    def calculate_bonus(self):
        pass

class Laptop:
    def __init__(self, brand):
        self.brand = brand

class Developer(Employee):
    def __init__(self, name, salary, language, laptop):
        super().__init__(name, salary)
        self.language = language
        self.laptop = laptop

    def calculate_bonus(self):
        return self.salary * 0.10

    def show_details(self):
        super().show_details()
```



```
print("Salary:", self.salary)
print("Language:", self.language)
print("Laptop:", self.laptop.brand)
print("Bonus:", self.calculate_bonus())

laptop = Laptop("Dell")

developer = Developer(
    name="Aamir",
    salary=50000,
    language="Python",
    laptop=laptop
)

developer.show_details()
```

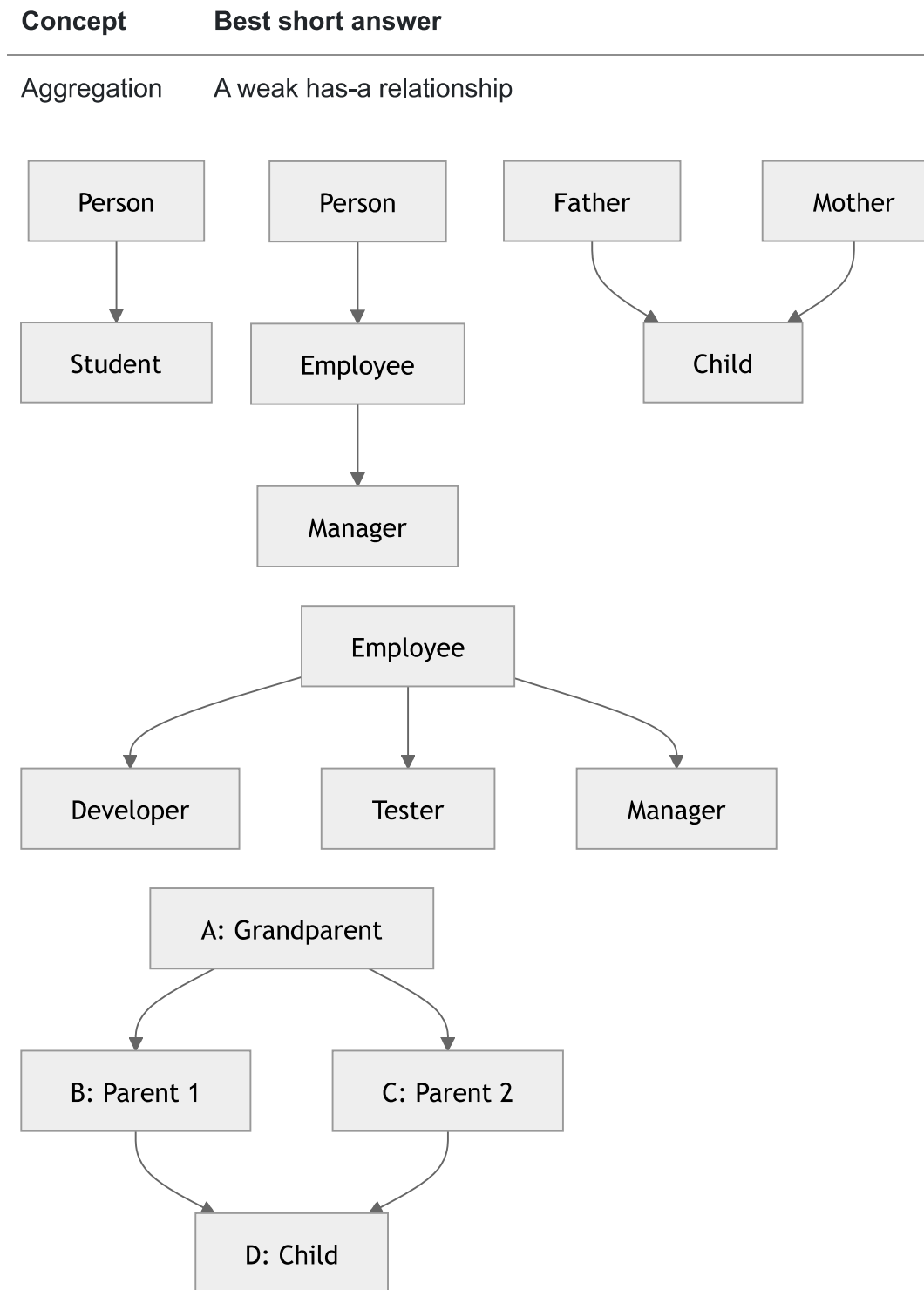
Concepts used:

Code part	OOP concept
Person, Employee, Developer	Classes
developer	Object
name, salary, language	Attributes
show_details()	Method
__init__()	Constructor
self	Current object
__salary	Encapsulation
Employee(Person)	Inheritance
Overridden show_details()	Polymorphism
Employee(ABC)	Abstraction
super().__init__()	Parent functionality
Developer → Employee → Person	MRO

Code part	OOP concept
Employee using Laptop	Aggregation

One-Line Interview Revision

Concept	Best short answer
Class	A blueprint that defines attributes and methods
Object	An actual instance created from a class
Attribute	Data stored in a class or object
Method	A function that defines object behaviour
Constructor	A special method that initializes an object
<code>self</code>	A reference to the current object
Encapsulation	Combining data and methods while controlling data access
Inheritance	Reusing and extending parent-class functionality
Polymorphism	The same interface producing different behaviours
Abstraction	Showing essential functionality while hiding implementation
<code>super()</code>	Accessing the next parent implementation according to MRO
MRO	The order Python follows to search inherited methods
Composition	A strong has-a relationship



You said:

all oops to design a graph and code and easy word materials

ChatGPT said:

Complete Python OOP — Graph, Code and Easy Study

Material

1. Complete OOP Concept Graph

```
mindmap
  root((OOP))
    Basics
      Class
      Object
      Attribute
        Instance attribute
        Class attribute
      Method
        Instance method
        Class method
        Static method
      Constructor
      self
    Four Pillars
      Encapsulation
        Public
        Protected
        Private
        Getter and Setter
        Property
      Inheritance
        Single
        Multilevel
        Multiple
        Hierarchical
        Hybrid
      Polymorphism
        Overriding
        Duck typing
        Operator overloading
        Overloading style
      Abstraction
        Abstract class
        Abstract method
        Concrete class
    Relationships
      Association
      Aggregation
      Composition
    Advanced
      super
      MRO
      Magic methods
      isinstance
```

```
issubclass  
dataclass
```

2. OOP Easy Meaning

OOP means **Object-Oriented Programming**.

It organizes programs using:

- Classes
- Objects
- Data
- Methods

Real-life example:

```
Class    → Car design  
Object   → Actual BMW car  
Attribute → Brand, colour and speed  
Method   → start(), stop() and drive()
```

3. Class and Object

Class

A class is a blueprint or plan for creating objects.

Object

An object is an actual instance created from a class.

```
class Student:  
    def study(self):  
        print("Student is studying")  
  
student1 = Student()  
student2 = Student()  
  
student1.study()  
student2.study()
```

Here:

- **Student** = class

- `student1` = first object
- `student2` = second object
- `study()` = method

Real-life example:

- Mobile design = class
- Your actual mobile = object

4. Constructor and `self`

Constructor

A constructor gives initial values to an object. In Python, `__init__()` is commonly called a constructor.

`self`

`self` represents the current object.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def introduce(self):
        print("My name is", self.name)
        print("My marks are", self.marks)

student1 = Student("Aamir", 90)
student2 = Student("Rahul", 85)

student1.introduce()
student2.introduce()
```

For `student1`, `self` means `student1`.

For `student2`, `self` means `student2`.

Constructor types

Non-parameterized constructor

```
class Student:
    def __init__(self):
        self.name = "Unknown"

student = Student()
print(student.name)
```

Parameterized constructor

```
class Student:
    def __init__(self, name):
        self.name = name

student = Student("Aamir")
```

Constructor with default values

```
class Student:
    def __init__(self, name="Unknown", marks=0):
        self.name = name
        self.marks = marks

student1 = Student()
student2 = Student("Aamir", 90)
```

5. Attributes

Attributes are variables that store object or class data.

Instance attribute

Different for every object.


```
class Student:
    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.name)
print(student2.name)
```

Class attribute

Shared by all objects.

```
class Student:
    college = "UEM Jaipur"

    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.college)
print(student2.college)
```

Instance attribute

Class attribute

Belongs to an object

Belongs to the class

Different for every object

Shared by all objects

Usually uses **self**

Written directly inside class

Instance attribute**Class attribute**

Example: student name

Example: college name

6. Types of Methods

flowchart TD

A["Methods"] --> B["Instance method"]

A --> C["Class method"]

A --> D["Static method"]

A --> E["Magic method"]

A --> F["Abstract method"]

Instance method

Works with object data and uses **self**.

```
class Student:
    def __init__(self, name):
        self.name = name

    def show_name(self):
        print(self.name)

student = Student("Aamir")
student.show_name()
```

Class method

Works with class data and uses **cls**.

```
class Student:
    college = "UEM Jaipur"

    @classmethod
    def change_college(cls, new_college):
        cls.college = new_college

Student.change_college("Delhi University")
print(Student.college)
```

Static method

A utility method that does not need **self** or **cls**.

```
class Calculator:
    @staticmethod
    def add(a, b):
        return a + b

print(Calculator.add(10, 20))
```

Method comparison

Method	First parameter	Works with
Instance method	self	Object data
Class method	cls	Class data
Static method	Nothing special	Utility logic

7. Four Pillars of OOP

```
flowchart TD
    A["Four OOP Pillars"] --> B["Encapsulation"]
    A --> C["Inheritance"]
    A --> D["Polymorphism"]
    A --> E["Abstraction"]
    B --> B1["Protect data"]
    C --> C1["Reuse code"]
    D --> D1["Different behaviours"]
    E --> E1["Hide internal logic"]
```

8. Encapsulation

Encapsulation means:

- Keep data and methods in one class.
- Protect important data.
- Control how data is changed.

Real-life example: A bank does not allow anyone to change account balance directly. Money must be changed using deposit or withdrawal operations.

Public attribute

Can be accessed from anywhere.

```
class Student:
    def __init__(self, name):
        self.name = name

student = Student("Aamir")
print(student.name)
```

Protected attribute

Uses one underscore `_`. It should normally be used inside the class and child classes.

```
class Employee:
    def __init__(self, salary):
        self._salary = salary
```

Python does not strictly block its access. The underscore is a programming convention.

Private attribute

Uses two underscores `__`.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def show_balance(self):
        return self.__balance

account = BankAccount(1000)
print(account.show_balance())
```

Direct access normally fails:

```
print(account.__balance) # AttributeError
```

Getter and setter

- Getter reads private data.

- Setter safely changes private data.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    def get_salary(self):
        return self.__salary

    def set_salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary")

employee = Employee(50000)

print(employee.get_salary())
employee.set_salary(60000)
print(employee.get_salary())
```

Property decorator

@property gives a cleaner getter and setter style.

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary")

employee = Employee(50000)

print(employee.salary)

employee.salary = 60000
print(employee.salary)
```

9. Inheritance

Inheritance allows a child class to reuse the parent class's code.

Real-life example:

```
Person
↓
Student
```

A student **is a** person.

```
class Person:
    def speak(self):
        print("Person can speak")

class Student(Person):
    def study(self):
        print("Student can study")

student = Student()
student.speak()
student.study()
```

Types of inheritance

```
flowchart TD
    A["Inheritance"] --> B["Single"]
    A --> C["Multilevel"]
    A --> D["Multiple"]
    A --> E["Hierarchical"]
    A --> F["Hybrid"]
```

Single inheritance

One child inherits from one parent.

```
class Person:
    def speak(self):
        print("Speaking")

class Student(Person):
    def study(self):
        print("Studying")
```

Multilevel inheritance

Inheritance happens through multiple levels.

flowchart TD
 A["Person"] --> B["Employee"]
 B --> C["Manager"]

```
class Person:
    def speak(self):
        print("Person speaks")

class Employee(Person):
    def work(self):
        print("Employee works")

class Manager(Employee):
    def manage(self):
        print("Manager manages")

manager = Manager()
manager.speak()
manager.work()
manager.manage()
```

Multiple inheritance

One child inherits from multiple parents.

flowchart TD
 A["Father"] --> C["Child"]
 B["Mother"] --> C

```
class Father:
    def drive(self):
        print("Can drive")

class Mother:
    def cook(self):
        print("Can cook")

class Child(Father, Mother):
    pass

child = Child()
child.drive()
child.cook()
```

Hierarchical inheritance

Multiple children inherit from one parent.

flowchart TD
A["Employee"] --> B["Developer"]
A --> C["Tester"]
A --> D["Manager"]

```
class Employee:
    def work(self):
        print("Employee works")

class Developer(Employee):
    pass

class Tester(Employee):
    pass
```

Hybrid inheritance

It combines two or more inheritance types.


```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```

10. **super()**

super() calls the next parent constructor or method according to MRO.

```
class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)

class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

    def show(self):
        super().show()
        print("Course:", self.course)

student = Student("Aamir", "AI/ML")
student.show()
```

Output:

```
Name: Aamir
Course: AI/ML
```

11. Polymorphism

Polymorphism means:

- **Poly** = many
- **Morphism** = forms
- Polymorphism = one interface with many behaviours

Real-life example: The same **pay()** operation can use UPI, card or cash.

Method overriding

A child class replaces a parent method.

```
class Animal:
    def sound(self):
        print("Animal sound")
```

```
class Dog(Animal):
    def sound(self):
        print("Dog barks")
```

```
class Cat(Animal):
    def sound(self):
        print("Cat meows")
```

```
Dog().sound()
Cat().sound()
```

Duck typing

Python cares about what an object can do, not only its class.

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

def make_sound(animal):
    animal.sound()

make_sound(Dog())
make_sound(Cat())
```

Any object containing `sound()` can be used.

Operator overloading

It changes how an operator behaves with custom objects.

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __str__(self):
        return f"₹{self.amount}"

money1 = Money(500)
money2 = Money(300)

print(money1 + money2)
```

Output:

```
₹800
```

Method overloading style

Python does not support traditional method overloading directly. We can use default parameters or `*args`.

```
class Calculator:
    def add(self, *numbers):
        return sum(numbers)

calculator = Calculator()

print(calculator.add(10, 20))
print(calculator.add(10, 20, 30))
```

12. Abstraction

Abstraction means showing necessary functionality and hiding internal logic.

Real-life example: We use ATM buttons without knowing the ATM's internal programming.

Abstract class and abstract method

```
from abc import ABC, abstractmethod

class Payment(ABC):

    @abstractmethod
    def pay(self, amount):
        pass

class UPI(Payment):

    def pay(self, amount):
        print(f"Paid ₹{amount} using UPI")

class Card(Payment):

    def pay(self, amount):
        print(f"Paid ₹{amount} using Card")

upi = UPI()
upi.pay(1000)
```

Important terms:

- **ABC** = Abstract Base Class
- Abstract class = common structure
- Abstract method = method a child must implement
- Concrete class = completed class that allows object creation
- **@abstractmethod** = marks a required method

This is invalid:

```
payment = Payment() # TypeError
```

13. Association

Association means two independent objects are connected.

Real-life example: A teacher teaches a student. Both can exist separately.

```
class Student:
    def __init__(self, name):
        self.name = name

class Teacher:
    def teach(self, student):
        print("Teaching", student.name)

student = Student("Aamir")
teacher = Teacher()

teacher.teach(student)
```

14. Aggregation

Aggregation is a weak **has-a** relationship. Both objects can exist separately.

Real-life example: A department has teachers. A teacher can exist without one particular department.

```
class Teacher:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, teacher):
        self.teacher = teacher

teacher = Teacher("Mr. Sharma")
department = Department(teacher)

print(department.teacher.name)
```

The teacher is created outside and passed to the department.

15. Composition

Composition is a strong **has-a** relationship. The contained object is created and controlled by the owner.

Real-life example: A car has an engine.

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
        print("Car started")

car = Car()
car.start()
```

The engine is created inside the car.

Object relationship graph

flowchart TD

A["Object Relationships"] --> B["Association"]

A --> C["Aggregation"]

A --> D["Composition"]

B --> B1["Objects are connected"]

C --> C1["Weak has-a"]

D --> D1["Strong has-a"]

Association vs aggregation vs composition

Relationship	Easy meaning	Real-life example
Association	Objects are connected	Teacher teaches Student
Aggregation	Weak has-a	Department has Teacher
Composition	Strong has-a	Car has Engine

16. MRO

MRO means **Method Resolution Order**.

It tells Python where to search for a method first.

```

class A:
    def show(self):
        print("Class A")

class B(A):
    pass

class C(A):
    def show(self):
        print("Class C")

class D(B, C):
    pass

obj = D()
obj.show()

print(D.mro())

```

Python searches in this order:

$D \rightarrow B \rightarrow C \rightarrow A \rightarrow \text{object}$

MRO is mainly important in multiple inheritance.

Python uses **C3 linearization** to create a consistent search order.

17. Magic Methods

Magic methods begin and end with double underscores. They are also called **dunder methods**.

Magic method	Purpose
<code>__init__()</code>	Initialize an object
<code>__str__()</code>	Return readable text
<code>__len__()</code>	Work with <code>len()</code>
<code>__add__()</code>	Work with <code>+</code>
<code>__eq__()</code>	Work with <code>==</code>

Magic method Purpose

`__new__()` Create an object

`__str__()`

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Student: {self.name}"

student = Student("Aamir")
print(student)
```

`__len__()`

```
class Team:
    def __init__(self, members):
        self.members = members

    def __len__(self):
        return len(self.members)

team = Team(["Aamir", "Rahul", "Priya"])
print(len(team))
```

`__eq__()`

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    def __eq__(self, other):
        return self.marks == other.marks

student1 = Student(90)
student2 = Student(90)

print(student1 == student2)
```

18. `isinstance()` and `issubclass()`

`isinstance()`

Checks whether an object belongs to a class.

```
class Student:
    pass

student = Student()

print(isinstance(student, Student))
```

`issubclass()`

Checks whether one class inherits from another.

```
class Person:
    pass

class Student(Person):
    pass

print(issubclass(Student, Person))
```

19. Identity vs Equality

- `is` checks whether two variables point to the same object.
- `==` checks whether their values are equal.

```
list1 = [10, 20]
list2 = [10, 20]
list3 = list1

print(list1 == list2) # True: same values
print(list1 is list2) # False: different objects
print(list1 is list3) # True: same object
```

20. Data Class

A data class automatically creates common methods such as `__init__()`, `__repr__()` and `__eq__()`.

```
from dataclasses import dataclass
```

```
@dataclass  
class Student:  
    name: str  
    course: str  
    marks: int
```

```
student = Student("Aamir", "AI/ML", 90)
```

```
print(student)
```

Output:

```
Student(name='Aamir', course='AI/ML', marks=90)
```

21. One Complete OOP Program

```
from abc import ABC, abstractmethod

class Person:
    organization = "Tech Solutions"

    def __init__(self, name):
        self.name = name

    def show_details(self):
        print("Name:", self.name)

class Employee(Person, ABC):
    def __init__(self, name, salary):
        super().__init__(name)
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary")

    @abstractmethod
    def calculate_bonus(self):
        pass

class Laptop:
    def __init__(self, brand):
        self.brand = brand

class Developer(Employee):
    def __init__(self, name, salary, language, laptop):
        super().__init__(name, salary)
        self.language = language
        self.laptop = laptop

    def calculate_bonus(self):
        return self.salary * 0.10
```

```

def show_details(self):
    super().show_details()
    print("Organization:", self.organization)
    print("Salary:", self.salary)
    print("Language:", self.language)
    print("Laptop:", self.laptop.brand)
    print("Bonus:", self.calculate_bonus())

laptop = Laptop("Dell")

developer = Developer(
    name="Aamir",
    salary=50000,
    language="Python",
    laptop=laptop
)

developer.show_details()

```

Concepts used in the program

Code	Concept
Person, Employee, Developer	Classes
developer	Object
name, salary, language	Attributes
show_details()	Method
__init__()	Constructor
self	Current object
__salary	Encapsulation
Employee(Person)	Inheritance
calculate_bonus()	Abstraction
Different method implementations	Polymorphism
super()	Parent functionality
Laptop passed to Developer	Aggregation

Code	Concept
Developer → Employee → Person	MRO

22. Complete Interview Revision Table

Concept	Easy interview answer
OOP	Programming using classes and objects
Class	Blueprint used to create objects
Object	Actual instance of a class
Attribute	Data associated with a class or object
Method	Function defined inside a class
Constructor	Initializes a new object
self	Represents the current object
Instance variable	Separate data for every object
Class variable	Data shared by all objects
Instance method	Works with object data
Class method	Works with class data
Static method	Utility method related to a class
Encapsulation	Protects and controls data access
Inheritance	Reuses and extends parent code
Polymorphism	Same interface with different behaviours
Abstraction	Hides implementation details
Overriding	Child replaces a parent method
Duck typing	Object behaviour matters more than its type
super()	Calls the next parent implementation
MRO	Order used to search inherited methods

Concept	Easy interview answer
Association	Two independent objects are connected
Aggregation	Weak has-a relationship
Composition	Strong has-a relationship
Magic method	Special method with double underscores
Data class	Class for storing data with less repeated code

Recommended Learning Order

flowchart TD

```
A["Class and Object"] --> B["Constructor and self"]
B --> C["Attributes and Methods"]
C --> D["Encapsulation"]
D --> E["Inheritance and super"]
E --> F["Polymorphism"]
F --> G["Abstraction"]
G --> H["Object Relationships"]
H --> I["MRO and Magic Methods"]
```

The most important interview topics are:

1. Class and object
2. Constructor and **self**
3. Instance variable vs class variable
4. Instance, class and static methods
5. Four pillars of OOP
6. Method overloading vs overriding
7. Inheritance types
8. **super()** and MRO
9. Abstraction vs encapsulation

10. Aggregation vs composition

